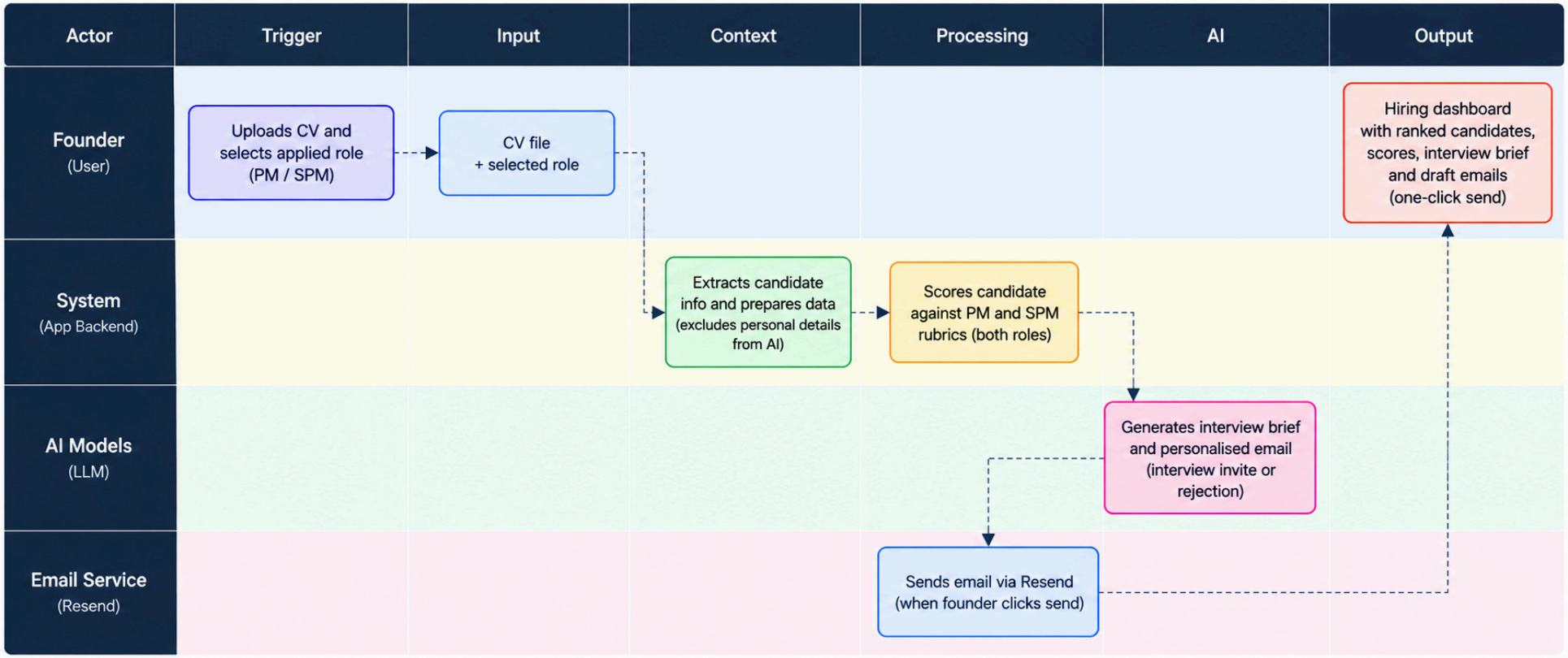
Case 2 · Hiring Dashboard — Components Map



CASE 2 · ARJUN - EXECUTION PLAN

L4 Sep 28

THE STACK

Tool	Role
Claude Chat	Rubric design - before any code is written
Claude Code	Where the project lives and gets built
Gemini Flash	All AI steps: extraction, scoring, brief, email draft
Supabase/Neon/Other	Database: candidates, scores, rubric criteria
Vercel/Netlify/Other	Hosts the dashboard and all routes
GitHub	Saves progress between sessions
Resend	Sends emails at the end of B

L4 · Monday 28 September · 60 Minutes

CHECKPOINT L4-1 · RUBRIC DESIGN IN CLAUDE CHAT

No code. You need: Claude open at claude.ai. The 8 files from the hires/ folder. The 2 files from the roles/ folder. The hire outcomes table from your case brief.

Upload all 8 hire profile documents and both role descriptions into a single Claude conversation. Then send this prompt:

> *"I've shared 10 documents with you. 8 are profiles of people who were hired at a company called Kargo - a logistics software startup in Mumbai. 2 are the job descriptions for the roles we are currently hiring for: Product Manager and Senior Product Manager.*
>
> *Here is a table showing the performance rating each of the 8 hires received:*
>
> *[copy and paste the table from your case brief here - all four columns]*
>
> *Use the job descriptions only to understand what the roles involve. Your rubric must not come from the job descriptions - it must come from the patterns you find in the hire profiles.*
>
> *Do the following in order:*

> *1. Compare the people rated Exceeds Expectations against those rated Meets Expectations or Below Expectations. What did the higher-rated people have in common - visible in their application or interview descriptions - that the others either lacked or showed only weakly? Find 2 to 3 patterns. Each pattern must come directly from what is written in the profiles. Do not use years of experience, job titles, or anything that the job descriptions already ask for.*
>
> *2. Using only those patterns as your source, design a hiring rubric for evaluating candidates applying for the Product Manager and Senior Product Manager roles at Kargo: between 4 and 6 criteria. For each criterion: give it a short name, write a plain-English description of what a strong candidate looks like on that criterion - specific enough that two people reading it would score the same CV the same way - and assign it a weight as a percentage. The PM and SPM roles may share the same criterion names, but the descriptions and weights can differ - the bar for SPM should be higher, especially on how independently the person has operated. Weights must add up to 100% for PM and 100% for SPM separately.*
>
> *3. Present the final rubric as two clear lists - one for PM, one for SPM - in this format for each criterion:*
> *Criterion name:*
> *What a strong candidate looks like:*
> *Weight:*
>
> *Do not include criteria that come from the job descriptions. If a criterion uses a phrase that sounds like it could appear in any job description, rewrite it until it is grounded in something specific from the profiles."*

When Claude responds, read the patterns in step 1 carefully before looking at the rubric. If any criterion sounds like a job description requirement, push back:

> *"The pattern you've described in [criterion name] reads like a job description requirement. Point to the specific person in the profiles whose application or interview is the source for this - what exactly did they say or do that you are trying to capture?"*

If any criterion description is still too vague to score a CV against, push back with:

> *"Rewrite the description for [criterion name] so that it is clear what a strong candidate looks like versus a weak one - in terms of what would actually appear in their work history."*

Before leaving Claude Chat, ask:

> *"Save this rubric as a plain text file called rubric.txt so I can download it."*

Download the file to your computer.

DONE WHEN: You have rubric.txt on your computer. It has 4 to 6 criteria for PM and 4 to 6 for SPM. Weights add up to 100% for each role. You can point to at least one hire profile that each criterion came from.

CHECKPOINT L4-2 · PROJECT SETUP IN CLAUDE CODE

Done by minute 60. You need: Claude Code open. Your rubric.txt file. Your Supabase/Neon/Other URL and anon key. Your Gemini API key.

Upload rubric.txt into Claude Code first. Then send this as your opening message:

> *"I'm building a hiring dashboard for a founder who is drowning in CVs and has made zero offers in eleven weeks.*
>
> *Here's how it works: the founder uploads a CV and selects which role the person applied for. The system reads the CV and immediately separates the candidate's personal details - name, email, phone - from everything else. The personal details are stored privately and never sent to any AI step. The rest of the CV goes through four steps automatically. First, a scoring engine evaluates the candidate against the hiring rubric and produces a score and a one-line reason for each criterion - every candidate is scored against both the PM and SPM rubric regardless of which role they applied for. Second, for the top candidates per role, the system generates a short three-sentence interview brief. Third, for every candidate, the system drafts a personalised email - an interview invite for those above the line, a warm rejection for those below it - written from the CV content, with the candidate's real name substituted from their stored personal details. Fourth, the founder opens the dashboard, sees all candidates ranked by score, reads the brief and draft email for each one, and clicks one button to send via Resend. Nothing goes out without him.*
>
> *I've attached a file called rubric.txt. This is the standard against which every candidate will be scored. It contains criteria for two roles - PM and SPM - each with a description and a weight. Read it, understand what it contains, and set up the database so the rubric is ready to use from the first scoring call. I have also attached the 2 JDs being hired for.*
>
> *Design philosophy: this is an internal tool for one person. Fast, clear, completely functional. No unnecessary design.*
>
> *Build the complete project - database, all pages, all scoring and brief and email draft logic, email sending via Resend, deployed on Vercel/Netlify/Other. I will share all the API keys next."*

Add your keys when Claude Code creates the .env.local file - replace each placeholder with your real Supabase/Neon/Other URL, Supabase/Neon/Other keys, and Gemini API key. Leave RESEND_API_KEY blank for now. Confirm .env.local is in .gitignore before pushing anything.

Deploy: push to GitHub, import into Vercel/Netlify/Other, add the same environment variables in Vercel/Netlify/Other settings, and deploy.

DONE WHEN: Your Vercel/Netlify/Other URL opens the upload page with no errors. The rubric_criteria table in Supabase/Neon/Other has rows in it - one per criterion per role, populated from rubric.txt. Push to GitHub before leaving the room. B starts from what you pushed.

B · Tuesday 29 September · 2 Hours · Build Session

The project is already built and deployed. The session is for uploading CVs, testing the pipeline, adding the Resend API key, and confirming the system works end to end.

DATA PRIVACY BLOCK

Not a build step. You have now designed a system that extracts, scores, and ranks people who never applied to Kargo. Three questions you should be able to answer before the build continues:

- What is the difference between the Gemini AI Studio free tier and the Gemini API with billing enabled, with respect to what happens to the data you send it?
- What does the extraction step in the pipeline do that would make it DPDP-compliant in a real production deployment?

CHECKPOINT B-1 · UPLOAD CVs AND TEST THE PIPELINE

You need: the 60 CVs from the applications/ folder.

Upload 3 CVs first - one clearly strong PM candidate, one clearly weak SPM candidate, and one that looks ambiguous. For each, select the correct role before uploading. Wait for the pipeline to complete and check:

- The candidates table in Supabase/Neon/Other has 3 rows. Open one row. The personal details field has a name, email, and phone. The CV content field has work experience and skills but no name, email, or phone visible anywhere in it.
- The scores table or score_json column has scores for both PM and SPM rubrics for each candidate.
- The dashboard shows the 3 candidates ranked by score with a score breakdown per criterion.
- The top candidate has a 3-sentence brief visible on their card.
- Every candidate has a draft email - an invite for the top one, a rejection for the others.

If anything is wrong, tell Claude Code what you see and ask it to fix it. Once the 3 CVs work correctly, upload all 60.

DONE WHEN: All 60 rows are in Supabase/Neon/Other with scores. The dashboard shows candidates ranked by score. The top 5 per role each have a brief and a draft invite email. Everyone else has a draft rejection.

CHECKPOINT B-2 · RESEND INTEGRATION

You need: a free Resend account. Your Resend API key.

Create a free account at resend.com if you have not already. Generate an API key from the Resend dashboard.

In Vercel/Netlify/Other environment variables, add RESEND_API_KEY with your real key. Redeploy the project.

Tell Claude Code:

> *"The RESEND_API_KEY is now set in Vercel/Netlify/Other. Wire up the Confirm button so that clicking it calls Resend to send the draft email. Use the candidate's email address from their stored personal details as the recipient - you will see the recipient as a MESA test address that appears in the candidate's email field; do not send to any real external address. Mark the candidate's record as sent once the email goes."*

DONE WHEN: Clicking Confirm on a candidate card triggers a Resend API call. The MESA test address receives the email within 30 seconds. The email body contains the candidate's real name, not [NAME]. The sent status updates in Supabase/Neon/Other.

CHECKPOINT B-3 · REVIEW UI AND END-TO-END TEST

Run the full end-to-end test:

- Upload 2 new CVs that have not been scored yet.
- Wait for the pipeline to complete automatically.
- Open the dashboard. Read the top candidate's brief. Read the draft email.
- Click Confirm on the top candidate.
- Check that the MESA test address receives the email within 30 seconds.
- Check that the candidate's card is marked as sent.

If anything in the review UI is unclear or hard to use, tell Claude Code what to fix. The dashboard is Arjun's tool - it should let him read, decide, and send without confusion.

DONE WHEN: The full loop - upload to send - runs without error in under 5 minutes. Push final code to GitHub.

WHAT TO USE WHEN

Task	Tool
Rubric design	Claude Chat - before any code exists
Project setup and all build steps	Claude Code
PII extraction from CVs	Gemini Flash - structured output, cheap per call
Rubric scoring (both roles)	Gemini Flash - fast, consistent output
Interview brief	Gemini Flash - synthesis from structured scores
Email drafting	Gemini Flash - tone-aware, candidate-specific
Database	Supabase/Neon/Other - free tier, persistent, queryable
Hosting	Vercel/Netlify/Other - connected from L4
Email delivery	Resend - free tier, one API call, production-grade